Toward a Generative Framework for Safe Scientific Software

Spencer Smith

Computing and Software Department
Faculty of Engineering
McMaster University

Mar 17, 2026

Favorite Thing About Stockholm – Fika!

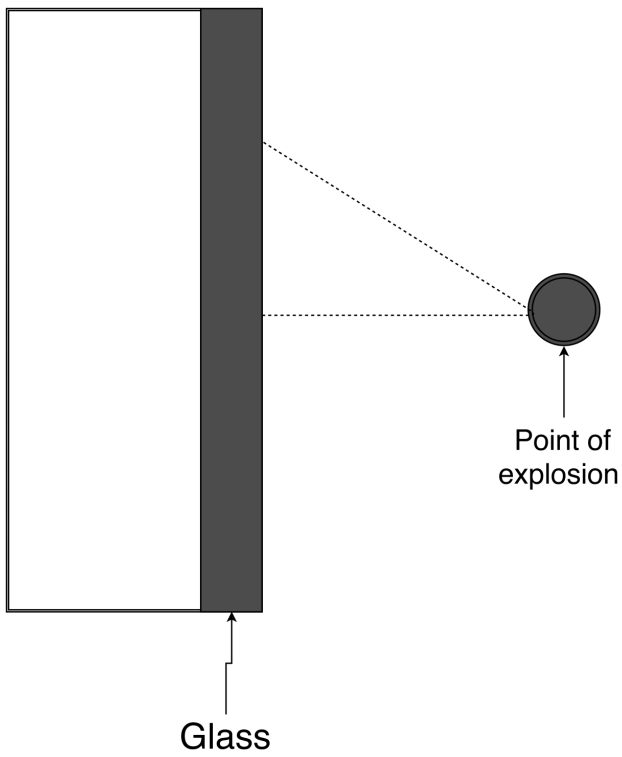
Toward a Generative Framework for Safe Scientific Software

1. Drasil – a gen framework for well-understood software
2. AC for computational medicine
3. Adding ACs to Drasil (work in progress)
   - Additional knowledge
   - Additional infrastructure

A Generative Framework for Scientific Software

- **Goal** – Improve qualities (e.g., safety, reliability, maintainability) of SS
- **Idea**
  - Adapt ideas from SE
  - Document requirements, design, verification, AC, etc.
    - Good – improves quality
    - Bad – too much work, inevitable change, too hard to maintain
- **Solution**
  - Capture knowledge once
  - Generate all documentation and code
- **Implement Solution – Drasil**
  - Facilitates change
  - Traceability
  - Reproducibility
  - Sustainability
  - Certification
  - Captures best practices

GlassBR



Given

- dimensions of glass plane
- glass type
- explosion characteristics
- tolerable breakage probability

Predict whether the glass will withstand the explosion

Drasil Inputs:

- Program Name: GlassBR
- Authors: Nikitha K and Spencer S
- Symbols: tolerable load ($\hat{q}_{\text{tol}}$), Risk of failure (B), ...
- Assumptions: Load duration factor constant,
- Data definitions: relation for B, ...
- Design decisions:
    - Modularity (input module),
    - Implementation Type (Program),
    - Logging (Yes),
    - Input Structure (Bundled),
    - Constant Structure (Inlined),
    - Constant Rep (Constants),
    - Real Number Rep (Double),
    - ...

Drasil Inputs:

- Program Name: GlassBR
- Authors: Nikitha K and Spencer S
- Symbols: tolerable load ($\hat{q}_{\text{tol}}$), Risk of failure (B), ...
- Assumptions: Load duration factor constant,
- Data definitions: relation for B, ...
- Design decisions:
    - Modularity (input module),
    - Implementation Type (Program),
    - Logging (Yes),
    - Input Structure (Bundled),
    - Constant Structure (Inlined),
    - Constant Rep (Constants),
    - Real Number Rep (Double),
    - ...

/glassbr
/Website/GlassBR_SRS.html
/Website/GlassBR_SRS.css
/SRS/bibfile.bib
/SRS/Makefile
/SRS/GlassBR_SRS.tex
/SRS/GlassBR_SRS.pdf
/src/python
/src/python/README.md
/src/python/InputParameters.py
/src/python/Calculations.py
/src/python/Makefile
/src/python/doxConfig
...

...
/src/java/GlassBR/Calculations.java
/src/java/Makefile
/src/java/README.md
...
/src/cpp/GlassBR
/src/cpp/ReadTable.cpp
/src/cpp/InputFormat.hpp
/src/cpp/Calculations.cpp
...
/src/swift/Calculations.swift
...
/src/csharp/Control.cs
...

/glassbr
/Website/GlassBR_SRS.html
/Website/GlassBR_SRS.css
/SRS/bibfile.bib
/SRS/Makefile
/SRS/GlassBR_SRS.tex
/SRS/GlassBR_SRS.pdf
/src/python
/src/python/README.md
/src/python/InputParameters.py
/src/python/Calculations.py
/src/python/Makefile
/src/python/doxConfig
...

...
/src/java/GlassBR/Calculations.java
/src/java/Makefile
/src/java/README.md
...
/src/cpp/GlassBR
/src/cpp/ReadTable.cpp
/src/cpp/InputFormat.hpp
/src/cpp/Calculations.cpp
...
/src/swift/Calculations.swift
...
/src/csharp/Control.cs
...

tex

Software Requirements Specification for GlassBR

Nikitha K and Spencer S

html

Table of Symbols

$\hat{q}_{\text{tol}}$

B

...

Introduction

... The software, herein called GlassBR, ...

Assumptions

ldfConstant: LDF is constant, depends on assumed value of t_d and m, ...

Data Definitions

$$B = \frac{k}{(ab)^{m-1}} \left(Eh^2\right)^m \text{LDF} e^J$$

...

README.md

GlassBR

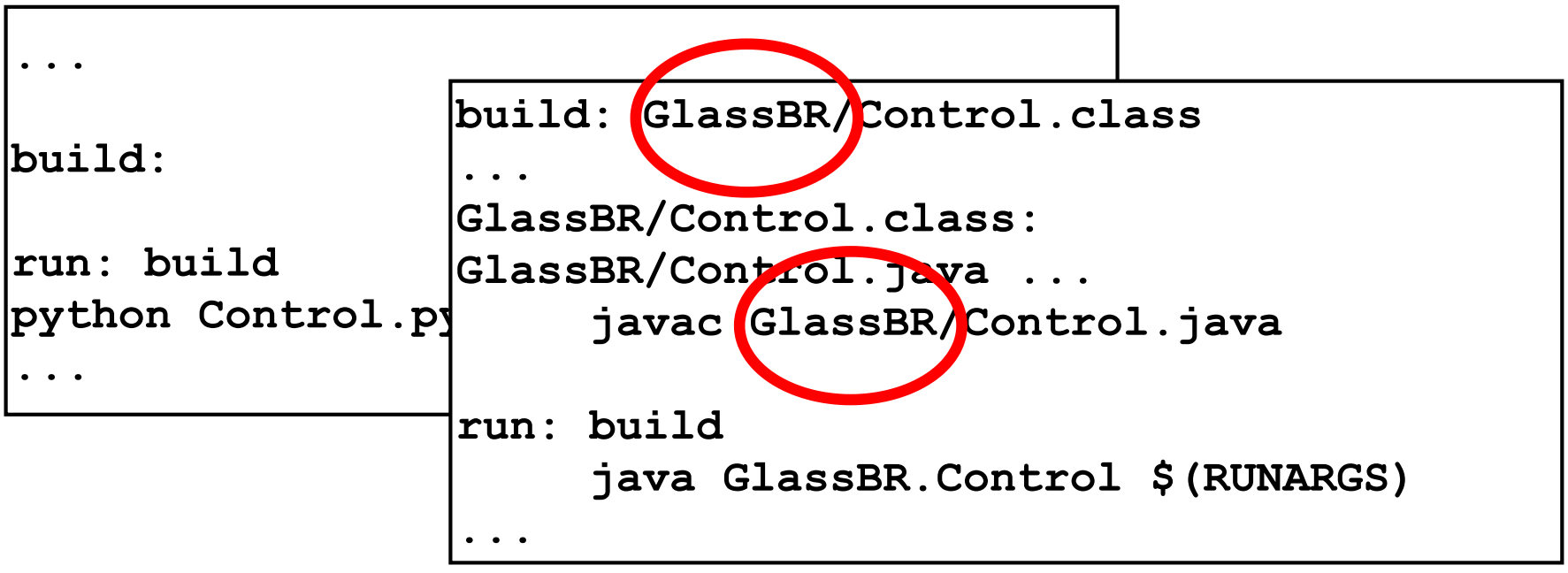
Authors Nikitha K and Spencer S

How to Run the Program: In your terminal command line, enter the same directory as this README file. Then enter the following line

```
make run RUNARGS=input.txt
```

Configuration Files: SDF.txt, TSD.txt must be in the same directory as the executable to run successfully

Versioning: Python Version 3.5.1


...
build:
run: build
python Control.py
...
build: GlassBR/Control.class
...
GlassBR/Control.class:
GlassBR/Control.java ...
javac GlassBR/Control.java
run: build
java GlassBR.Control $(RUNARGS)
...

Calculations.py

```
## \file Calculations.py
# \author Nikitha Krithnan and W. Spencer Smith
# \brie
...
## \bri
# \para
# \para
# \retu
def fur
    out
    pri
    ...
    out

    ret
```

Calculations.java

```
package GlassBR;
/** \file Calculations.java
    \author Nikitha Krithnan and W. Spencer Smith
    \brief Provides functions for calculating the outputs
*/
...
    public static double func_B(InputParameters inParams, double J) throws IOException {
      PrintWriter outfile;
      outfile = new PrintWriter(new FileWriter(new File("log.txt"), true));
      outfile.println("function func_B called with inputs: {");
      ...
      outfile.close();

      return 2.86e-53 /Math.pow(inParams.a * inParams.b, 7.0 - 1.0) *
             Math.pow(7.17e10 * Math.pow(inParams.h, 2.0), 7.0) * inParams.LDF
             * Math.exp(J);
    }
```

J_{tol} in SRS.pdf

Refname	**DD:sdfTol**
Label	Stress distribution factor (Function) based on Pbtol
Symbol	J_{tol}
Units	Unitless
Equation	$$J_{\text{tol}} = \ln\left(\ln\left(\frac{1}{1-P_{\text{btol}}}\right)\frac{\left(\frac{a}{1000}\frac{b}{1000}\right)^{m-1}}{k\left(E\cdot 1000\left(\frac{h}{1000}\right)^2\right)^m LDF}\right)$$
Description	J_{tol} is the stress distribution factor (Function) based on Pbtol (Unitless) P_{btol} is the tolerable probability of breakage (Unitless) a is the plate length (long dimension) (m) b is the plate width (short dimension) (m) m is the surface flaw parameter $(\frac{\text{m}^{12}}{\text{N}^7})$ k is the surface flaw parameter $(\frac{\text{m}^{12}}{\text{N}^7})$ E is the modulus of elasticity of glass (Pa) h is the minimum thickness (m) LDF is the load duration factor (Unitless)

J_{tol} in SRS.tex

```
...
Label & Stress distribution factor (Function) based on Pbtol

\\ \midrule \\
Symbol & ${J_{\text{tol}}}$

\\ \midrule \\
Units & Unitless

\\ \midrule \\
Equation & \begin{displaymath}
           {J_{\text{tol}}}=\ln\left(\ln\left(\frac{1}{1-{P_
               {\text{b}\text{tol}}}}\right) \frac{\left(\
               frac{a}{1000} \frac{b}{1000}\right)^{m-1}}{k \
               left(E\cdot{}1000 \left(\frac{h}{1000}\right)
               ^{2}\right)^{m} LDF}\right)
           \end{displaymath}
\\ \midrule \\
Description & ...
```

J_{tol} in SRS.html

```
...
<th>Equation</th>
<td>
\[{J_{\text{tol}}}=\ln\left(\ln\left(\frac{1}{1-{P_{\text{b}}\
    text{tol}}}}\right) \frac{\left(\frac{a}{1000} \frac{b
    }{1000}\right)^{m-1}}{k \left(E\cdot{}1000 \left(\frac{h
    }{1000}\right)^{2}\right)^{m} LDF}\right)\]
</td>
...
```

J_{tol} in Python

```
## \brief Calculates stress distribution factor (Function)
    based on Pbtol
# \param inParams structure holding the input values
# \return stress distribution factor (Function) based on
    Pbtol
def func_J_tol(inParams):
    outfile = open("log.txt", "a")
    print("function func_J_tol called with inputs: {", file=
        outfile)
    print("  inParams = ", end="", file=outfile)
    print("Instance of InputParameters object", file=outfile)
    print("  }", file=outfile)
    outfile.close()

    return math.log(math.log(1.0 / (1.0 - inParams.P_btol)) *
         ((inParams.a / 1000.0 * (inParams.b / 1000.0)) **
        (7.0 - 1.0) / (2.86e-53 * (7.17e10 * 1000.0 * (
        inParams.h / 1000.0) ** 2.0) ** 7.0 * inParams.LDF)))
```

J_{tol} in Java

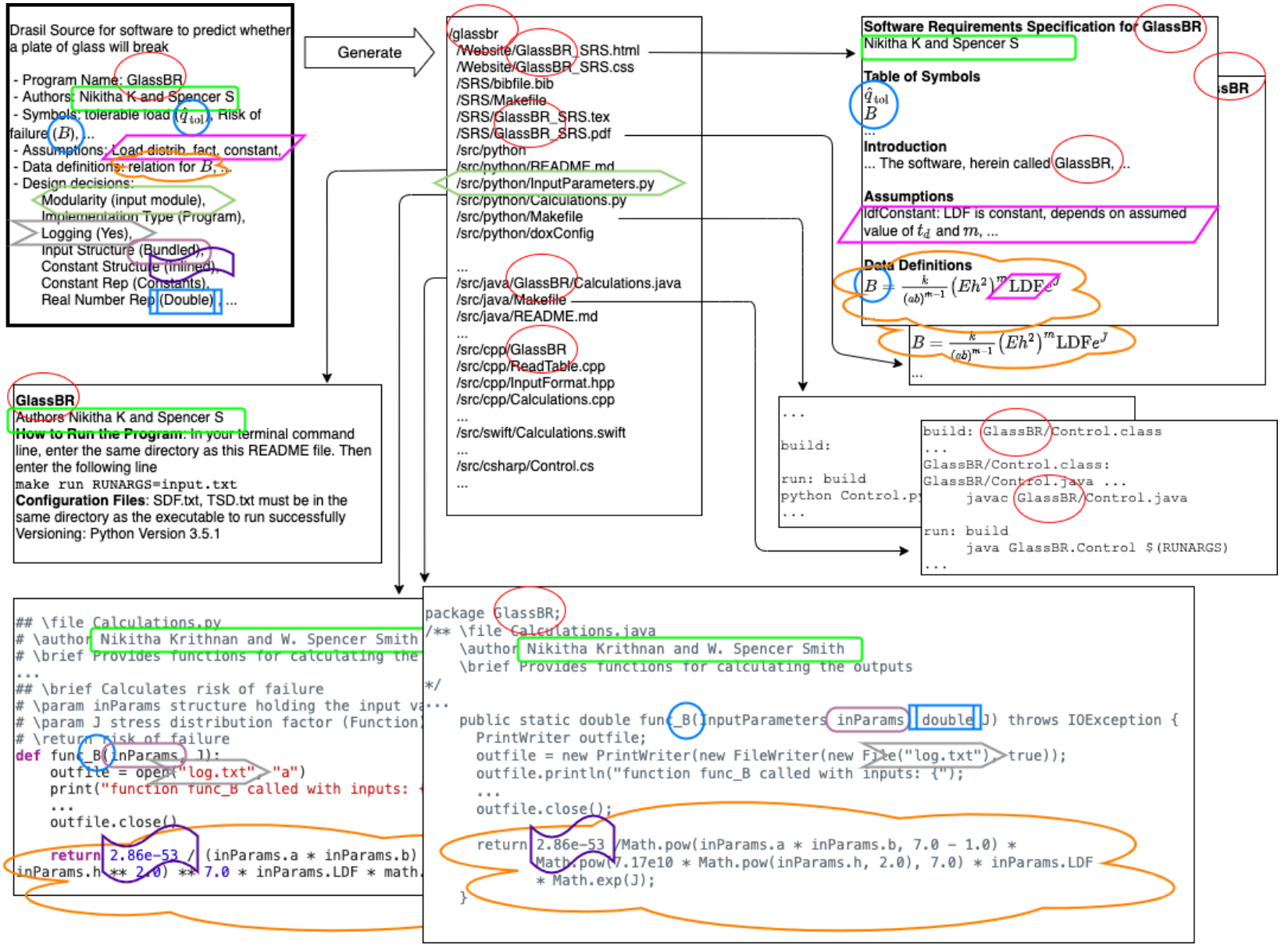
```
/** \brief Calculates stress distribution factor (
    Function) based on Pbtol
    \param inParams structure holding the input values
    \return stress distribution factor (Function) based
        on Pbtol
*/
public static double func_J_tol(InputParameters inParams)
     throws IOException {
    PrintWriter outfile;
    outfile = new PrintWriter(new FileWriter(new File("
        log.txt"), true));
    ...
    return Math.log(Math.log(1.0 / (1.0 - inParams.P_btol
        )) * (Math.pow(inParams.a / 1000.0 * (inParams.b
        / 1000.0), 7.0 - 1.0) / (2.86e-53 * Math.pow(7.17
        e10 * 1000.0 * Math.pow(inParams.h / 1000.0, 2.0)
        , 7.0) * inParams.LDF)));
}
```

J_{tol} in Drasil (Haskell)

```
tolStrDisFacEq :: Expr
tolStrDisFacEq = ln (ln (recip_ (exactDbl 1 $- sy pbTol))
  `mulRe` (((sy plateLen $/ exactDbl 1000) `mulRe` (sy
     plateWidth $/ exactDbl 1000)) $^ (sy sflawParamM $-
     exactDbl 1) $/
    (sy sflawParamK `mulRe` ((sy modElas `mulRe` exactDbl
       1000 `mulRe`
    square (sy minThick $/ exactDbl 1000)) $^ sy sflawParamM)
         `mulRe` sy lDurFac)))
```

J_{tol} without Unit Conversion

```
tolStrDisFacEq :: Expr
tolStrDisFacEq = ln (ln (recip_ (exactDbl 1 $- sy pbTol))
  `mulRe` ((sy plateLen `mulRe` sy plateWidth) $^ (sy
     sflawParamM $- exactDbl 1) $/
    (sy sflawParamK `mulRe` ((sy modElas `mulRe`
    square (sy minThick)) $^ sy sflawParamM) `mulRe` sy
       lDurFac)))
```


Drasil Source for software to predict whether a plate of glass will break
- Program Name: GlassBR
- Authors: Nikitha K and Spencer S
- Symbols: tolerable load ($\hat{q}_{tol}$), Risk of failure (B), ...
- Assumptions: Load distrib. fact. constant, ...
- Data definitions: relation for B, ...
- Design decisions:
Modularity (input module),
Implementation Type (Program),
Logging (Yes),
Input Structure (Bundled),
Constant Structure (Inlined),
Constant Rep (Constants),
Real Number Rep (Double), ...
Generate
/glassbr
/Website/GlassBR_SRS.html
/Website/GlassBR_SRS.css
/SRS/bibfile.bib
/SRS/Makefile
/SRS/GlassBR_SRS.tex
/SRS/GlassBR_SRS.pdf
/src/python
/src/python/README.md
/src/python/InputParameters.py
/src/python/Calculations.py
/src/python/Makefile
/src/python/doxConfig
...
/src/java/GlassBR/Calculations.java
/src/java/Makefile
/src/java/README.md
...
/src/cpp/GlassBR
/src/cpp/ReadTable.cpp
/src/cpp/InputFormat.hpp
/src/cpp/Calculations.cpp
...
/src/swift/Calculations.swift
...
/src/csharp/Control.cs
...
Software Requirements Specification for GlassBR
Nikitha K and Spencer S
Table of Symbols
$\hat{q}_{tol}$
B
...
Introduction
... The software, herein called GlassBR, ...
Assumptions
ldfConstant: LDF is constant, depends on assumed value of t_d and m, ...
Data Definitions
$B = \frac{k}{(ab)^{m-1}} (Eh^2)^m \text{LDF} e^J$
$B = \frac{k}{(ab)^{m-1}} (Eh^2)^m \text{LDF} e^J$
...
GlassBR
Authors Nikitha K and Spencer S
How to Run the Program: In your terminal command line, enter the same directory as this README file. Then enter the following line
make run RUNARGS=input.txt
Configuration Files: SDF.txt, TSD.txt must be in the same directory as the executable to run successfully
Versioning: Python Version 3.5.1
...
build:
run: build
python Control.py
...
build: GlassBR/Control.class
...
GlassBR/Control.class:
GlassBR/Control.java ...
javac GlassBR/Control.java
run: build
java GlassBR.Control $(RUNARGS)
...
\file Calculations.py
\author Nikitha Krithnan and W. Spencer Smith
\brief Provides functions for calculating the
...
\brief Calculates risk of failure
# \param inParams structure holding the input va
# \param J stress distribution factor (Function)
# \return risk of failure
def func_B(inParams, J):
outfile = open("log.txt", "a")
print("function func_B called with inputs:
...
outfile.close()
return 2.86e-53 / (inParams.a * inParams.b)
inParams.h ** 2.0) ** 7.0 * inParams.LDF * math.
package GlassBR;
/** \file Calculations.java
\author Nikitha Krithnan and W. Spencer Smith
\brief Provides functions for calculating the outputs
*/
...
public static double func_B(InputParameters inParams, double J) throws IOException {
PrintWriter outfile;
outfile = new PrintWriter(new FileWriter(new File("log.txt"), true));
outfile.println("function func_B called with inputs: {");
...
outfile.close();
return 2.86e-53 /Math.pow(inParams.a * inParams.b, 7.0 - 1.0) *
Math.pow(7.17e10 * Math.pow(inParams.h, 2.0), 7.0) * inParams.LDF
* Math.exp(J);
}

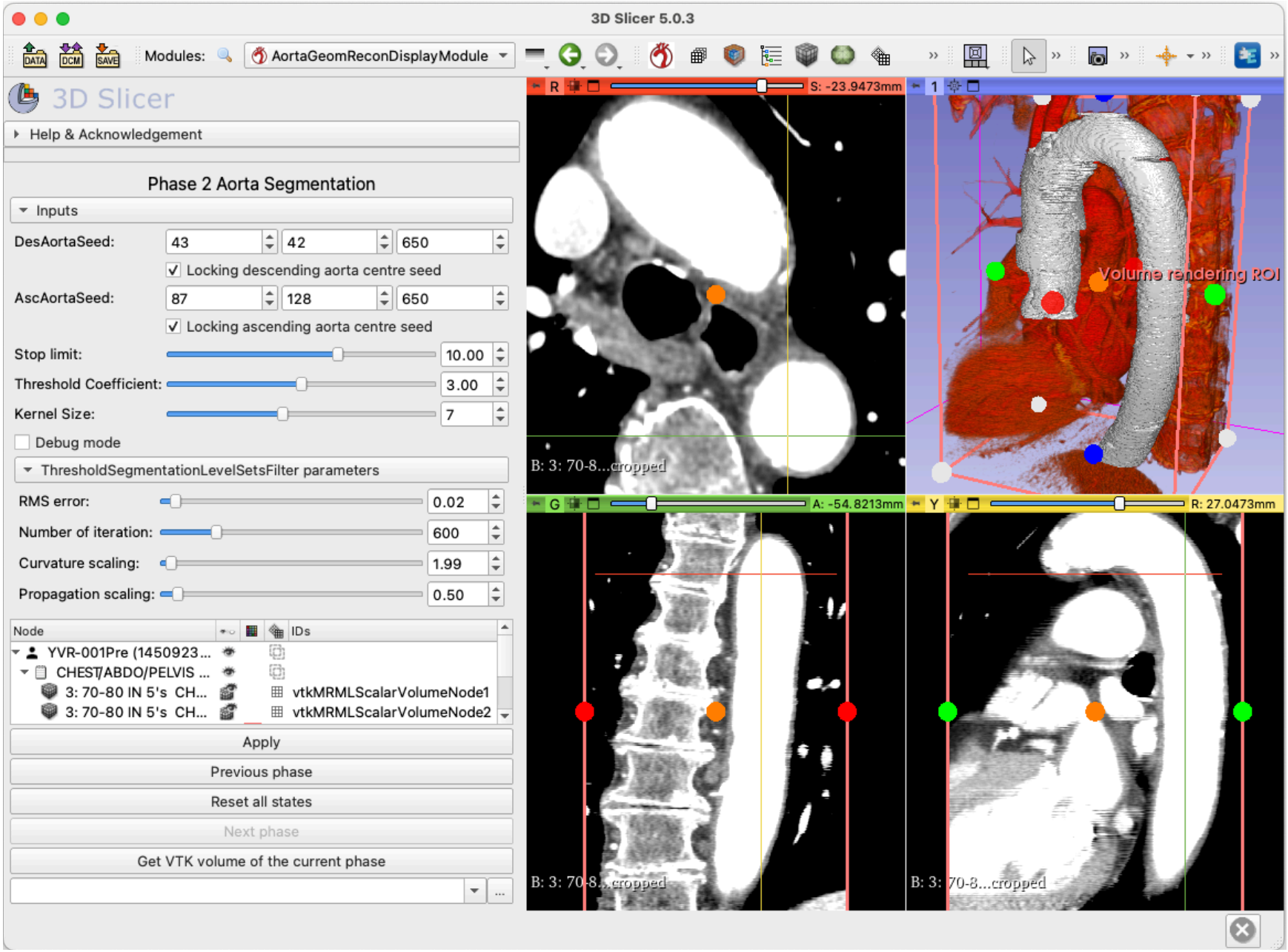
What Drasil Can Currently Do

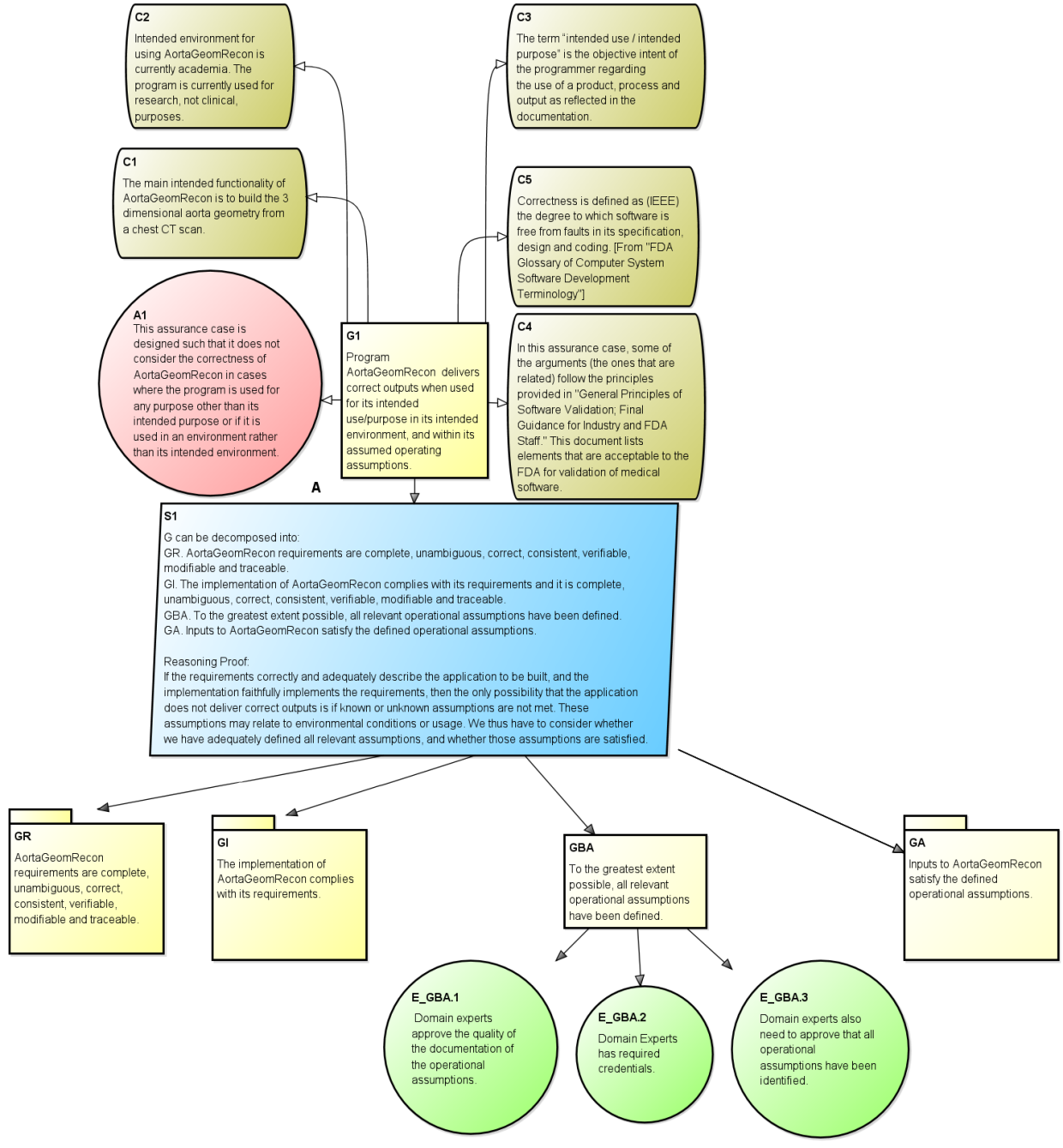
- Explicit relations, LHS = ...
- Systems of higher-order ODEs
- Knowledge on rigid body mechanics, heat transfer
- SRS (LaTeX, html, MD books)
- Code (Python, C++, C sharp, Java, Swift), Jupyter notebooks
- README files, Makefile
- Doxygen comments
- Infrastructure for knowledge about people

Advantages of AC Generation

- Build AC and software together
- Facilitates change
- Traceability
- Reuse of common knowledge
- Design space exploration

AC Example for Computational Medicine

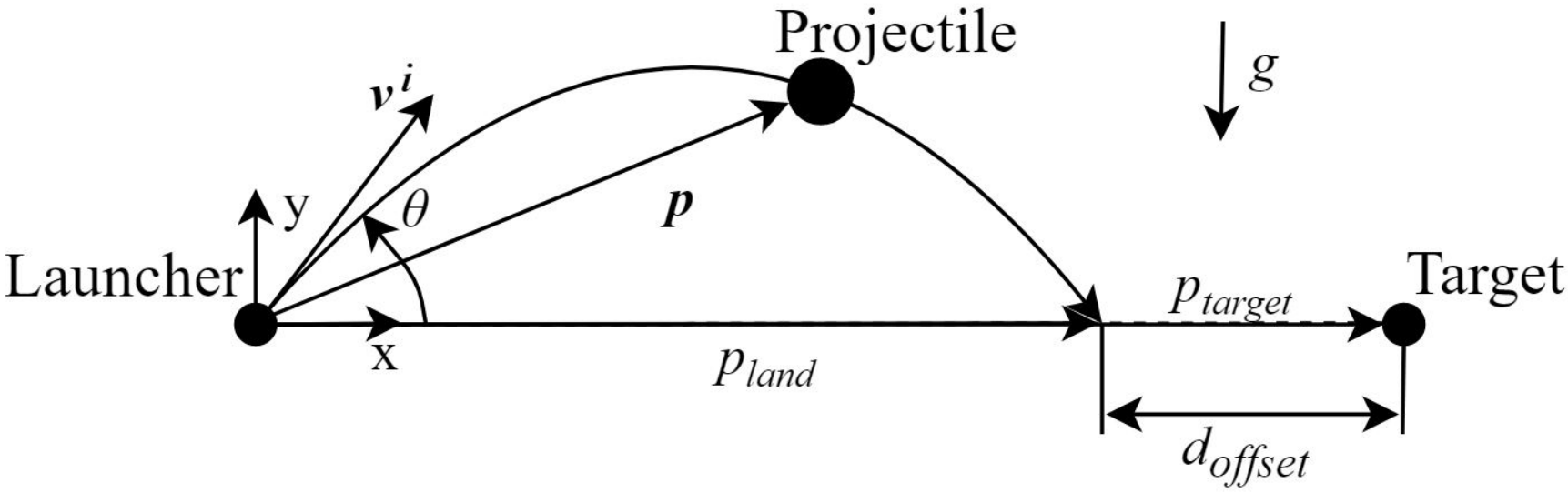



C2
Intended environment for using AortaGeomRecon is currently academia. The program is currently used for research, not clinical, purposes.
C3
The term "intended use / intended purpose" is the objective intent of the programmer regarding the use of a product, process and output as reflected in the documentation.
C1
The main intended functionality of AortaGeomRecon is to build the 3 dimensional aorta geometry from a chest CT scan.
C5
Correctness is defined as (IEEE) the degree to which software is free from faults in its specification, design and coding. [From "FDA Glossary of Computer System Software Development Terminology"]
A1
This assurance case is designed such that it does not consider the correctness of AortaGeomRecon in cases where the program is used for any purpose other than its intended purpose or if it is used in an environment rather than its intended environment.
G1
Program AortaGeomRecon delivers correct outputs when used for its intended use/purpose in its intended environment, and within its assumed operating assumptions.
C4
In this assurance case, some of the arguments (the ones that are related) follow the principles provided in "General Principles of Software Validation; Final Guidance for Industry and FDA Staff." This document lists elements that are acceptable to the FDA for validation of medical software.
A
S1
G can be decomposed into:
GR. AortaGeomRecon requirements are complete, unambiguous, correct, consistent, verifiable, modifiable and traceable.
GI. The implementation of AortaGeomRecon complies with its requirements and it is complete, unambiguous, correct, consistent, verifiable, modifiable and traceable.
GBA. To the greatest extent possible, all relevant operational assumptions have been defined.
GA. Inputs to AortaGeomRecon satisfy the defined operational assumptions.
Reasoning Proof:
If the requirements correctly and adequately describe the application to be built, and the implementation faithfully implements the requirements, then the only possibility that the application does not deliver correct outputs is if known or unknown assumptions are not met. These assumptions may relate to environmental conditions or usage. We thus have to consider whether we have adequately defined all relevant assumptions, and whether those assumptions are satisfied.
GR
AortaGeomRecon requirements are complete, unambiguous, correct, consistent, verifiable, modifiable and traceable.
GI
The implementation of AortaGeomRecon complies with its requirements.
GBA
To the greatest extent possible, all relevant operational assumptions have been defined.
GA
Inputs to AortaGeomRecon satisfy the defined operational assumptions.
E_GBA.1
Domain experts approve the quality of the documentation of the operational assumptions.
E_GBA.2
Domain Experts has required credentials.
E_GBA.3
Domain experts also need to approve that all operational assumptions have been identified.

Required Additions to Drasil for ACs

- Additional Knowledge
  - Generation of GSN diagrams
  - Reviewer reports
  - Test case generation
- Additional infrastructure
  - Rationale/evidence for decisions and assumptions
  - Theories and theory refinement

Projectile Motion Example



(WIP) SRS.pdf

Validity of Model I

- Scope Decisions
  - Orientation of projectile ignored
  - Only kinematic equations (no forces)
- Modelling Decisions
  - Cartesian coordinate system
  - y axis opposite to gravity
  - Launcher at origin
  - Target on x axis
  - Positive x direction from launcher to target
  - Time starts at zero
  - Launch velocity is positive
  - Magnitude and angle rep. for initial velocity
- Data constraints
  - Inputs
    - $p_{\text{target}} > 0$
    - $v_{\text{launch}} > 0$
    - $0 < \theta < \frac{\pi}{2}$
    -

Validity of Model II

- Outputs
  - $p_{\text{land}} > 0$
  - $d_{\text{offset}} > -p_{\text{target}}$
  - $t_{\text{flight}} > 0$
- Helper Assumptions
  - The motion of the particle is one dimensional
  - The acceleration is constant
- Generic Assumptions
  - The variables only depend on two dimensions
  - The acceleration is constant in the x direction
  - The acceleration is constant in the y direction
  - The acceleration is due to gravity
- Projectile Specific Assumptions
  - The acceleration in the x-direction is zero
  - The constant acceleration in the y-direction is due to gravity
  - The constant acceleration in the y-direction is -9.8 m/s^2
  - There are no obstructions to impede the projectile's motion
  - The surface of the planet is flat (no curvature)

Validity of Model III

- Rationale Assumptions
  - Only care about the motion of the centre of mass of the projectile
  - Distance is small enough that curvature can be neglected
  - Air drag is neglected
  - The rotation of the planet is neglected

Concluding Remarks

- Generate assurance cases for models, numerical algorithms and code implementation
- Add ACs to Drasil
- Modelling decisions/assumptions
  - Scope decisions
  - Modelling decisions
  - Data constraints
  - Assumptions
- Justify model via an AC
  - Rationale
  - Evidence (possibly via generation of test cases)
- Justify numerical algorithm via an AC
- Little theories, theory refinement
- Propagation of modelling decisions/assumptions through theories
- Measure and regenerate